

ICLAD 2026 GENAI CHIP HACKATHON • NVIDIA • CLOUD TRACK

Verification-Gated RTL Optimization

Optimize aggressively with an LLM — **ship only what's proven correct**. A Gemini agent rewrites RTL for power/performance/area; a formal + simulation gate refuses any rewrite that changes behavior.

TEAM T16 • MOONSHOT • UC SANTA CRUZ

Correctness is the gate. PPA is the prize. Hidden tests are the judge.

- ◆ Use an LLM agent to optimize IP RTL — `async_fifo`, `sha512`, `NVDLA`, `OpenTitan AES` / `ASCON` / `PRIM` / `KMAC`.
- ◆ **Two-phase scoring:** functional correctness first → then PPA (Yosys / OpenSTA) + LLM calls & token cost.
- ◆ **Winners judged at DAC on hidden testcases** — passing the visible testbench is not enough.

Consequence

An "optimization" that looks smaller/faster but is subtly wrong scores **zero** — and hidden tests are built to find it. So correctness-under-uncertainty is the real competition.

The failure mode that loses the hackathon is a **silent** functional regression.

- ◆ We proved it's real: a candidate with **lower area that passed simulation but was formally inequivalent** — and the reverse.
- ◆ **Thesis:** make **formal equivalence a hard gate** inside the optimization loop.
- ◆ Then the agent can chase PPA hard, because the gate — not the model — decides what ships.

PPA-only optimizer

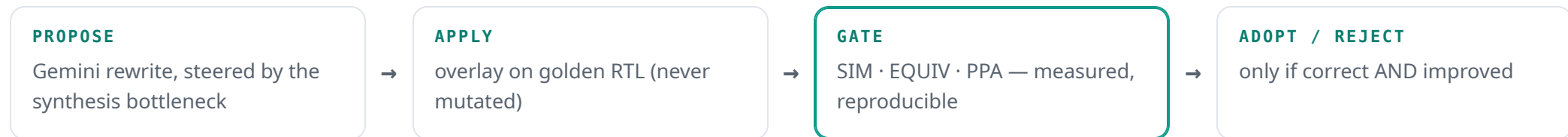
Fast. Passes the visible bench. Ships silent bugs → fails hidden tests.

Verification-gated

Fast *and* safe. Aggressive rewrites, zero accepted regressions.

SYSTEM ARCHITECTURE

Model proposes. The gate decides. Loop.



↳ feedback each round: **bottleneck breakdown** + **rejection memory** (what failed & why)

The loop is deterministic Python; the model only proposes. Adoption is decided by the gate's measured verdict, never the model's claim.

Transformation-driven, not "make it smaller."

- ◆ Sees the real **synthesis bottleneck** — sequential vs combinational area, dominant cell types.
- ◆ Handed a **transformation menu** grounded in RTLRewriter / ROVER / ASPEN (register, FSM, resource-sharing, operator-restructuring, width, boolean, strength).
- ◆ Told **not to repeat** what Yosys + ABC already do; keeps **rejection memory** across rounds.
- ◆ Emits surgical edits *or* whole-file rewrites; records **LLM calls + tokens** (the scored cost).

```
# prompt (abridged)
objective: minimize area
bottleneck: seq 13% · comb 87%
  top: XOR2×227, INV×90, OAI21×54
menu: T1 reg · T4 operator-restructuring ...
rejected so far: (none)
return: {edits|files}, equivalence-preserving
```

Three gates, one verdict.

SIM

The scored correctness check — SVUT / Icarus / plain-Verilog TBs. Hard gate.

EQUIV

Yosys-native sequential equivalence (bundled SAT + ABC — no external solver, runs in the eval Docker as-is).

PPA

Yosys synth + OpenSTA on ASAP7 — area, WNS, worst-slack, power, plus a cell-level bottleneck.

Timing = relative

Reject only on WNS *regression* — designs can start timing-negative (sha512 -169 ps).

```
// verdict.json { "sim":true, "equiv":{"verified":true}, "ppa":{"area_um2":89.4,"wns_ps":-9.0}, "accept":true }
```

XOR/parity datapaths are SAT-hard — so we tier the evidence, honestly.

proven

formal proof converges → highest confidence

sim-verified

formal inconclusive → directed + 20k-vector miter passes

refuted

miter finds a mismatch → hard reject

- ◆ Measured: the CRC "flatten" defeated **both** Yosys-native equiv **and** ABC SAT-sweeping CEC (each > 6 min timeout).
- ◆ Confidence level travels with every verdict — we don't pretend everything is proven.
- ◆ Future: XOR-aware EC (eqy / CryptoMiniSat) or our GF(2) linear checker to reclaim proven.

ONE FRAMEWORK, MANY IPS

Config-driven. Add an IP with a JSON file.

IP	LANGUAGE	SIM	SYNTH FLOW	EQUIV
async_fifo	Verilog	SVUT	run_syn.sh	proven
sha512	Verilog	Icarus TB	run_syn.sh	proven
prim_crc32	SystemVerilog	verilator pre-DV	syn.tcl + sv2v	proven

3 IPs · Verilog + SystemVerilog · 3 sim frameworks · 2 synth flows · sv2v — the harness generalizes; adding an IP is config + (if needed) a small sim fixture, not a rewrite.

The gate has teeth.

REWRITE	SIM	EQUIV	DECISION
inverted empty-flag (async_fifo)	fail	fail	rejected
dropped XOR term (crc32 flatten)	pass	refuted	rejected
CRC linearize (correct)	pass	proven	adopted

Row 2 is the key case: the functional testbench **passed** a broken rewrite (it exercises a different width), and the **equivalence miter** caught it — why we don't rely on the given TB alone. No functionally-broken rewrite was ever adopted, at any tier.

RESULTS · PPA WINS (GATED, CORRECTNESS-PRESERVING)

Every number here survived the correctness gate.

IP · OBJECTIVE	BASELINE → BEST	Δ	CONFIDENCE
prim_crc32 · area	92.35 → 89.40 μm ²	-3.19%	proven
prim_crc32 · timing	WNS -282 → -9 ps	97% closed	sim-verified

A datapath (CRC) rewrite: modest area win at **proven** confidence, and a near-timing-closing win at **sim-verified** confidence — both correctness-preserving through the gate.

A hard, high-value rewrite — correctly gated.

- ◆ **Bottleneck:** 4-deep **serial** byte-CRC chain (~14 logic levels) → WNS -282 ps.
- ◆ **Rewrite (ASPEN-style):** collapse the linear recurrence into one depth~5 **GF(2) XOR network**.
- ◆ **Result:** WNS **-282** → **-9 ps** (timing essentially closed), area +59% (tradeoff), 5000-vector miter **PASS**.
- ◆ Formal equiv timed out (XOR-hard) → accepted **sim-verified**; a 1-term corruption → **refuted**.

serial (baseline)		~14 lvl
flattened		~5 lvl
WNS closed		97%

Reproducible: `gen_crc_flat.py` → `run_gate.py` (repo README). The agent's autonomous --objective timing rewrite is a separate, formally-**proven** -282 → -270 ps win.

We operate the official eval environment — and fixed it.

- ◆ Found & fixed **two real bugs** in the official Dockerfile: Yosys config-clang removed at HEAD → pinned v0.50; Verilator VERILATOR_ROOT misinstall.
- ◆ ASAP7 techlib + sv2v wired; every run emits **verdict + agent-summary JSON**.
- ◆ One-command gate and agent loop; public repo with full instructions.

```
# reproduce
docker build -t iclad-dev:v1 .
python3 run_gate.py --config designs/prim_crc32.json \
  --candidate opentitan/hw/ip/prim/rtl
python3 opt_agent.py --backend vertex \
  --objective area --iters 8
```

github.com/Divya2030/iclad2026-moonshot-rtl-opt-agent

COST & EFFICIENCY

Token & call cost is a first-class metric.

- ◆ The loop **records usage** (LLM calls, input/output tokens) every run and adopts **only on measured improvement** — no churn.
- ◆ Levers: bottleneck-targeted prompts (fewer wasted turns), rejection memory (no repeats), synth caching per candidate.
- ◆ Adoption is gated on a **measured PPA delta**, so tokens are only spent turning proposals into verified wins — cost scales with results, not attempts.

The scoring counts calls/tokens; we treat efficiency as designed-in, not incidental.

What's proven vs assumed — stated plainly.

Formal ceiling on XOR-heavy rewrites

Mitigated by the tiered miter (sim-verified < proven; residual risk quantified). Fix: XOR-aware EC or the GF(2) linear checker.

Coverage

3 of 7 IPs deeply validated; framework generalizes to the rest via config.

Autonomy

The Vertex/Gemini proposer is implemented (`opt_agent --backend vertex`); the wins shown were human-seeded to validate the gate first. The autonomous end-to-end run is the next step.

Metric

Official PPA weighting still pending — strategy stays general (correctness-first, then Pareto).

KEY TAKEAWAYS

Verify, so you can optimize hard.

- ◆ **Verification-gated optimization** — aggressive PPA search that cannot ship a functional regression → built for hidden-test judging.
- ◆ **Tiered, honest confidence** — proven → sim-verified → refuted, instead of pretending everything is proven.
- ◆ **Reusable, config-driven framework** — 3 IPs, Verilog + SystemVerilog, two synth flows, in the official Docker.
- ◆ **Real, gated wins** — prim_crc32 area -3.19% (proven) and timing -282 → -9 ps (sim-verified), correctness-preserving.

T16 MOONSHOT · UC SANTA CRUZ — THANK YOU